The Recycling Portfolio

Momentum Timing Across Bitcoin, Copper, and Natural Gas

Robin Langer

August 2026

"When one asset says 'sit out,' put the money where another asset says 'get in.'"

$10,000 invested five years ago ⟶ $108,003 today.

Why These Three?

The daily strategy returns of the three assets have near-zero correlation:

	BTC	Copper	NatGas
BTC	1.000	—	—
Copper	0.056	1.000	—
NatGas	0.017	−0.032	1.000

Near-zero correlation means that when one asset is active, the others usually aren't. Over the 5-year period, all three are active simultaneously on only **2% of days**. This is portfolio diversification at its best: the assets take turns using the capital.

The Three Assets

Each asset was selected from a broader cross-asset study (see *The Bookmaker's Edge*) based on three criteria: strong momentum alpha, reliable volume data, and low correlation with the other assets.

Bitcoin (BTC-USD, 180-day lookback)

The flagship. The bookmaker model produces its strongest and most consistent signal on Bitcoin.

Metric	Value
Strategy return	+462.7%
Buy & hold return	+167.5%
Alpha	**+295.2%**
Sharpe ratio	1.16
Max drawdown	−28.1% (vs −76.6% buy & hold)
Win rate	62% (16 trades)
Win/loss ratio	14:1 (+26.8% avg win vs −1.9% avg loss)
Days in asset	34%

The Sharpe ratio measures return per unit of risk: average return divided by its volatility, annualised. Above 1.0 is strong; it means each unit of uncertainty is rewarded with more than one unit of return.

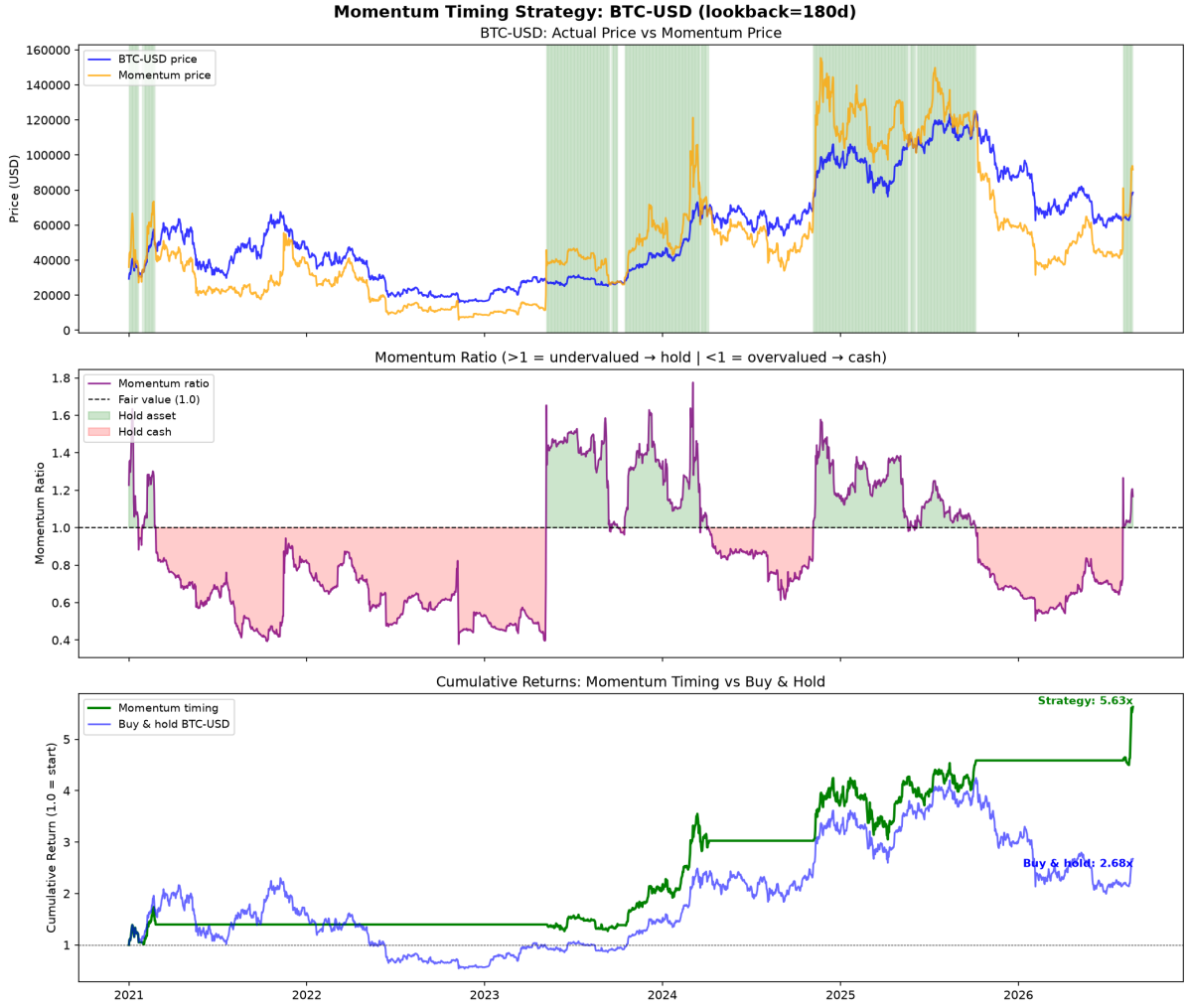


Figure 1: Bitcoin: the model exits before the 2022 crash, re-enters during the 2023 accumulation, and rides the 2024–2025 bull run.

How to read the charts

Each asset has a three-panel chart:

1. **Top — Price vs Momentum Price.** Blue = actual price. Orange = *momentum price*, a synthetic price from volume-weighted flow. Green shading marks days when the model says "hold the asset."
2. **Middle — Momentum Ratio.** Momentum price ÷ actual price. Above the dashed line at 1.0: **undervalued** (hold asset). Below: **overvalued** (hold cash). The further from 1.0, the stronger the signal.
3. **Bottom — Cumulative Returns.** Green = strategy, blue = buy & hold. Where green is above blue, the strategy is outperforming. Dips are drawdowns — temporary losses before recovery.

Copper (HG=F, 180-day lookback)

The quiet achiever. Copper has the highest Sharpe ratio among commodity futures and a perfect win rate.

Metric	Value
Strategy return	+97.0%
Buy & hold return	+82.0%
Alpha	+15.0%
Sharpe ratio	1.05
Max drawdown	−23.1% (vs −34.9% buy & hold)
Win rate	100% (6 trades)
Avg win	+12.3%
Days in asset	35%

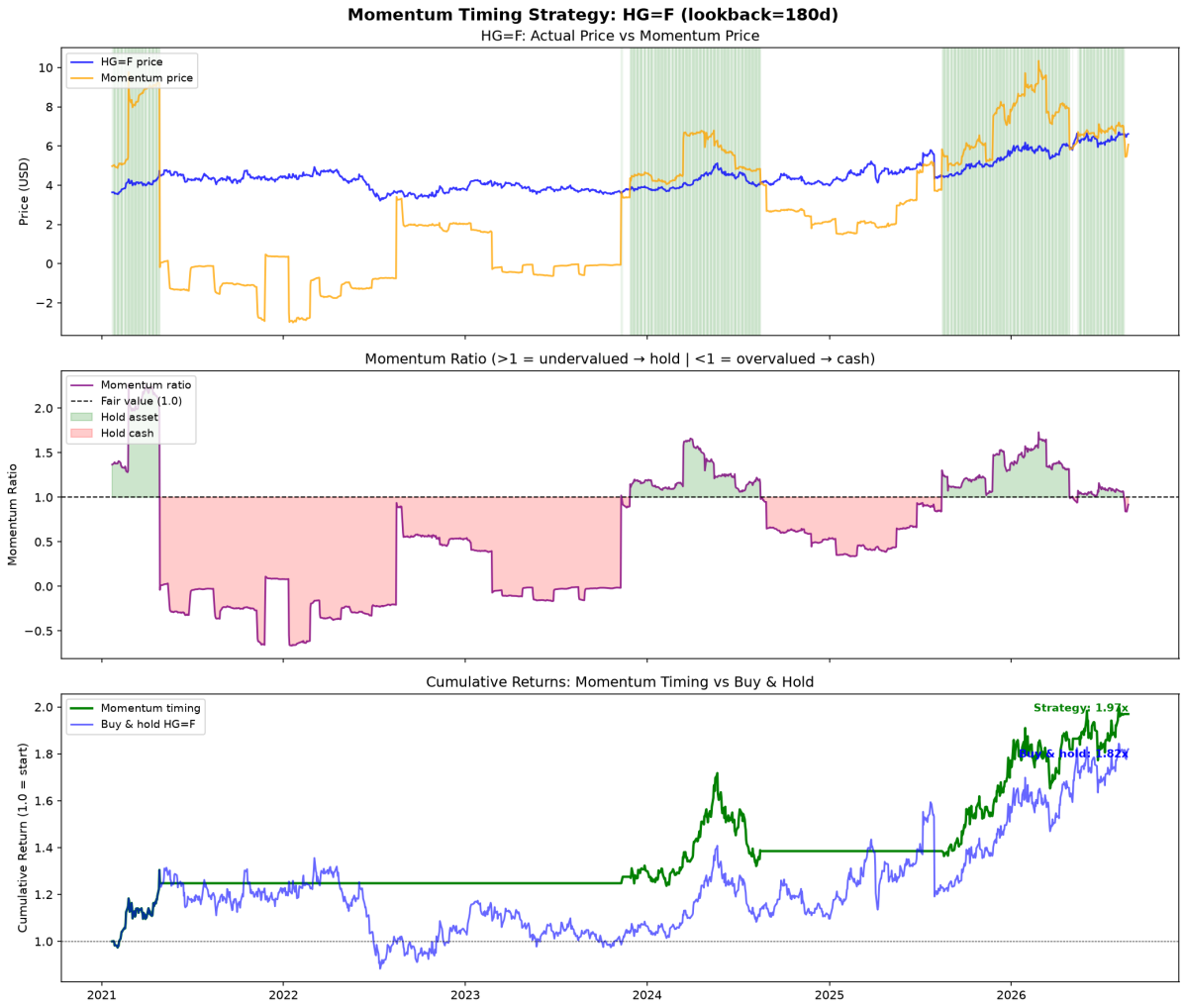


Figure 2: Copper: steady outperformance with minimal drawdown. The model catches the 2024–2025 commodity rally while avoiding the 2022 tightening-driven decline.

Natural Gas (NG=F, 90-day lookback)

The wildcard. Natural gas is the most volatile commodity tested, and the *only* asset where all three lookbacks (90, 180, 365 days) produce positive alpha. The 90-day lookback is optimal.

Metric	**Value**
Strategy return	+309.9%
Buy & hold return	+10.7%
Alpha	**+299.2%**
Sharpe ratio	0.90
Max drawdown	−53.0% (vs −83.7% buy & hold)
Win rate	53% (17 trades)
Win/loss ratio	4.5:1 (+25.6% avg win vs −5.7% avg loss)
Days in asset	50%

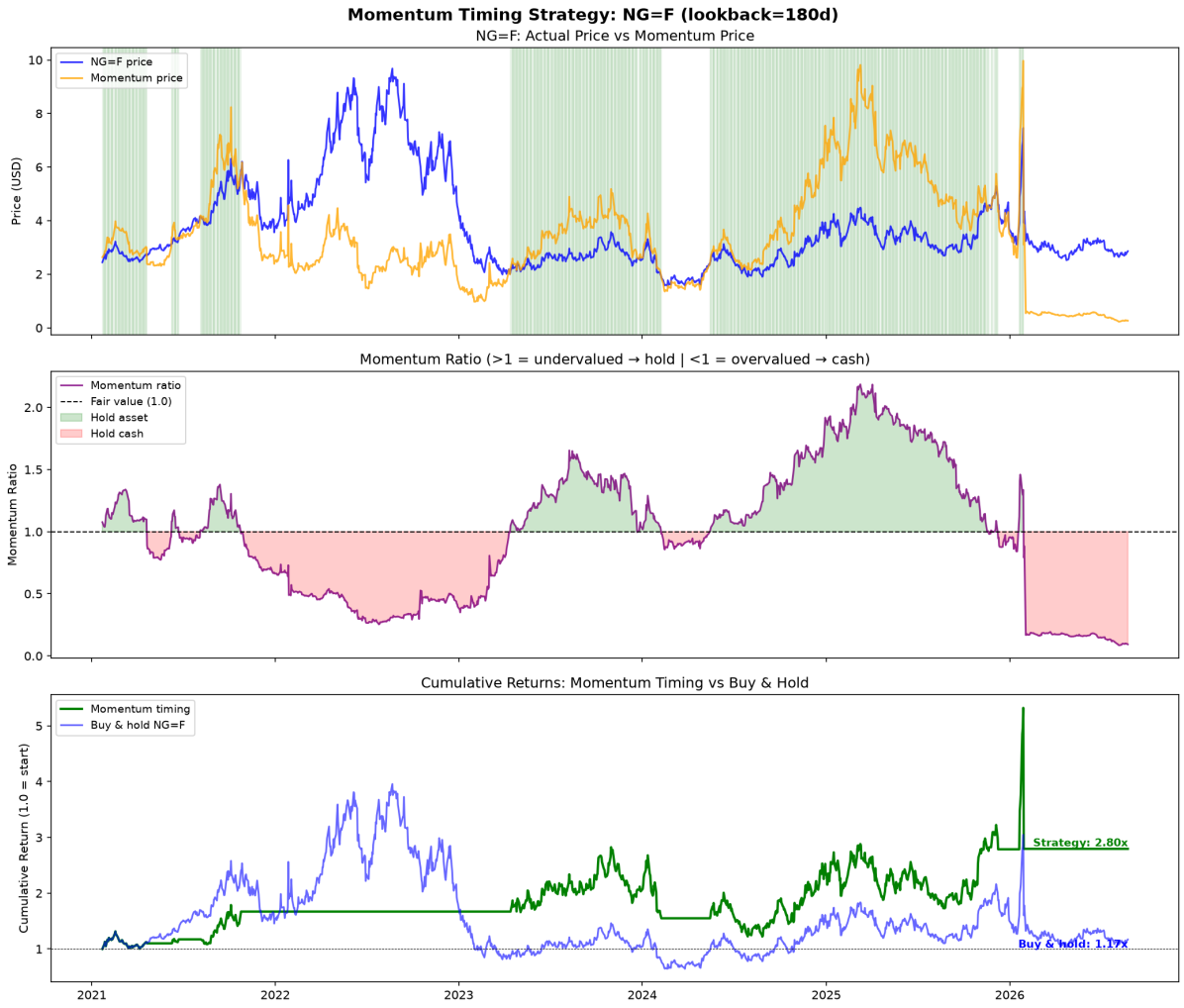


Figure 3: Natural gas: the model captures the 2021–2022 energy crisis spike and exits before the 83% crash that followed. Buy & hold returned only +11% over 5 years; the strategy returned +310%.

The Capital Recycling Idea

Pool all $10,000 and allocate dynamically based on which assets have active momentum signals:

Active assets	Allocation
3 assets active	33% / 33% / 33%
2 assets active	50% / 50%
1 asset active	100% in that asset
0 assets active	100% cash

When Bitcoin's signal says "go to cash" but copper's signal says "hold," the money that *would* have sat idle flows into copper instead.

Results

Period: 22 January 2021 to 24 August 2026 (1,406 trading days). Starting capital: $10,000.

Metric	Value
Final value	**$108,003**
Total return	**+980%**
Sharpe ratio	1.28
Max drawdown	−35.3%

Capital Utilisation

State	Days	% of time
0 assets active (100% cash)	368	26%
1 asset active	429	31%
2 assets active	580	41%
3 assets active	28	2%
Capital deployed	**1,037**	**74%**

Compare this to individual assets, which are deployed only 34–50% of the time. The recycling portfolio has capital working **74% of the time**.

Portfolio Equity Curve

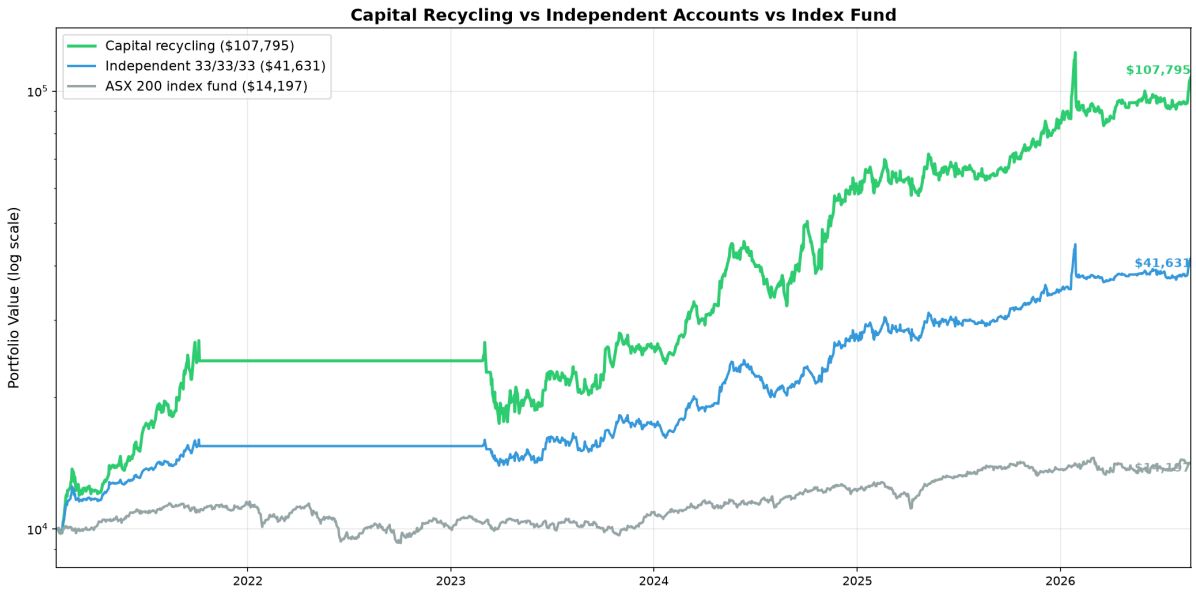


Figure 4: Capital recycling vs running three independent accounts vs the ASX 200 index fund (log scale). Recycling turns $10,000 into $108,003 by keeping idle capital deployed.

Caveats

1. **Backtest ≠ live trading.** These results assume execution at the daily close with no slippage, fees, or bid-ask spreads.
2. **Small sample sizes.** 6–17 trades per asset over 5 years is not statistically robust. Strong alpha may be driven by a few large wins.

3. **No transaction costs.** Crypto exchange fees (0.1–0.5%), futures commissions, and the bid-ask spread on commodity futures will reduce returns. The recycling model rebalances frequently, amplifying this cost.
4. **Tax drag.** Each trade is a taxable event. Frequent rebalancing creates short-term capital gains.
5. **Volume data quality.** Crypto volumes may be inflated by wash trading. Commodity futures volumes from yfinance are a fraction of actual CME volumes.
6. **Execution complexity.** The recycling model requires simultaneous positions in crypto (24/7 markets) and commodity futures (exchange hours only). Rebalancing across asset classes adds operational risk.
7. **Survivor bias.** We selected these three assets *because* they showed strong momentum alpha. An out-of-sample test is essential before committing real capital.

Historical returns do not guarantee future returns.

The results in this report are based on backtested simulations using historical data from January 2021 to August 2026. Past performance is not indicative of future results. Market conditions, volume dynamics, and correlations between assets can and do change. The bookmaker-pool model may stop working if the volume–price dynamics that produced these signals evolve.

Before committing real capital, paper trade the strategy for at least 3–6 months to verify that the signals remain actionable in live market conditions.

Source Code

All code lives in ~/git/andy/crypto/. Python 3.8+, no exchange account needed for backtesting.

Setup

Shell

```
cd ~/git/andy/crypto
python3 -m venv venv
source venv/bin/activate
pip install yfinance pandas numpy matplotlib
```

Running the Individual Backtests

Shell

```
# Bitcoin (180-day lookback)
python momentum_single.py --asset BTC-USD \
       --lookback 180 --start 2021-01-01 --plot

# Copper (180-day lookback)
python momentum_single.py --asset HG=F \
       --lookback 180 --start 2021-01-01 --plot

# Natural Gas (90-day lookback)
python momentum_single.py --asset NG=F \
       --lookback 90 --start 2021-01-01 --plot
```

momentum_single.py

The individual asset backtests. Implements the bookmaker-pool momentum model and the timing simulation.

momentum_single.py

```
"""
Crypto momentum timing: single-asset directional
strategy.

Uses the bookmaker-pool momentum model to time
entries and exits. Hold when momentum price > actual
price; go to cash when momentum price < actual price.
```

```
8
9   Usage:
10    python momentum_single.py
11    python momentum_single.py --asset HG=F
12    python momentum_single.py --lookback 90 --plot
13  """
14
15  import argparse, os
16  import numpy as np
17  import pandas as pd
18  import yfinance as yf
19  from datetime import datetime, timedelta
20  import matplotlib
21  matplotlib.use("Agg")
22  import matplotlib.pyplot as plt
23
24  SCRIPT_DIR = os.path.dirname(
25      os.path.abspath(__file__))
26
27
28  def compute_rolling_momentum(history, lookback_days):
29      """
30      Rolling momentum price via the bookmaker pool.
31
32      Volume is normalised (divided by window mean) to
33      prevent overflow on crypto's enormous volumes.
34      """
35      closes = history["Close"].values
36      volumes = history["Volume"].values
37      dates = history.index
38
39      momentum_prices = [np.nan] * len(history)
40      momentum_ratios = [np.nan] * len(history)
41
42      for i in range(lookback_days, len(history)):
43          window_close = closes[
44              i - lookback_days:i + 1]
45          window_vol = volumes[
46              i - lookback_days:i + 1]
47
48          if len(window_close) < 10:
49              continue
50
51          returns = (np.diff(window_close)
52                     / window_close[:-1])
53          vols = window_vol[1:]
54
55          mean_vol = vols.mean()
56          if mean_vol == 0:
57              continue
58          norm_vols = vols / mean_vol
59
60          # Net flow = normalised_volume * return
61          net_flows = norm_vols * returns
62
63          # Run the bookmaker pool
64          pool = float(window_close[0])
65          mp = pool
66          for j in range(len(net_flows)):
67              pool += net_flows[j] * mp
68              mp = pool
69
```

```
        momentum_prices[i] = mp
        momentum_ratios[i] = mp / closes[i]

    result = pd.DataFrame({
        "Close": closes,
        "MomentumPrice": momentum_prices,
        "MomentumRatio": momentum_ratios,
    }, index=dates)

    return result.dropna()


def simulate_timing(asset_symbol, lookback_days,
                    start_date, end_date):
    """
    Simulate the timing strategy.

    Rule:
      MomentumRatio > 1.0 -> hold asset
      MomentumRatio <= 1.0 -> hold cash
    """
    fetch_start = start_date - timedelta(
        days=lookback_days + 60)

    ticker = yf.Ticker(asset_symbol)
    hist = ticker.history(
        start=fetch_start, end=end_date,
        interval="1d")

    if hist.empty:
        raise ValueError(
            f"No data for {asset_symbol}")

    df = compute_rolling_momentum(
        hist, lookback_days)

    # Trim to simulation period
    start_ts = pd.Timestamp(start_date)
    if df.index.tz is not None:
        start_ts = start_ts.tz_localize(
            df.index.tz)
    df = df[df.index >= start_ts]

    # Signals: 1 = hold asset, 0 = hold cash
    df["signal"] = (
        df["MomentumRatio"] > 1.0).astype(int)

    # Position: use previous day's signal
    df["position"] = (
        df["signal"].shift(1).fillna(0).astype(int))

    # Returns
    df["asset_return"] = (
        df["Close"].pct_change().fillna(0))
    df["strategy_return"] = (
        df["position"] * df["asset_return"])

    # Cumulative
    df["cumulative_asset"] = (
        1 + df["asset_return"]).cumprod()
    df["cumulative_strategy"] = (
        1 + df["strategy_return"]).cumprod()
```

```
    return df
```

portfolio_recycling.py

The three-asset recycling portfolio simulation.

portfolio_recycling.py

```
"""
Three-asset recycling portfolio.

When an asset's momentum signal says "cash", its
capital flows to whichever other assets are active.
"""

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from datetime import datetime
from momentum_single import simulate_timing

ASSETS = [
    ("BTC-USD", 180),  # (ticker, lookback_days)
    ("HG=F",   180),   # copper
    ("NG=F",    90),   # natural gas
]
START = datetime(2021, 1, 1)
END   = datetime(2026, 8, 25)
CAPITAL = 10_000


def run_portfolio():
    # Run individual backtests
    sims = {}
    for ticker, lookback in ASSETS:
        sims[ticker] = simulate_timing(
            ticker, lookback, START, END)

    # Align to common dates (futures calendar)
    futures = [
        t for t, _ in ASSETS if "USD" not in t]
    common = sims[futures[0]].index
    for t in futures[1:]:
        common = common.intersection(
            sims[t].index)

    aligned = {}
    for ticker, _ in ASSETS:
        if "USD" in ticker:
            aligned[ticker] = sims[
                ticker].reindex(
                    common, method="ffill")
        else:
            aligned[ticker] = sims[
                ticker].loc[common]

    n_days = len(common)
```

```
    n_assets = len(ASSETS)

    signals = np.column_stack([
        aligned[t]["position"].values
        for t, _ in ASSETS])
    returns = np.column_stack([
        aligned[t]["asset_return"].values
        for t, _ in ASSETS])

    # Simulate recycling portfolio
    portfolio = np.ones(n_days)
    daily_rets = np.zeros(n_days)

    for i in range(1, n_days):
        active = [
            j for j in range(n_assets)
            if signals[i, j] == 1]
        if not active:
            daily_rets[i] = 0.0
        else:
            w = 1.0 / len(active)
            port_ret = sum(
                w * returns[i, j]
                for j in active)
            daily_rets[i] = port_ret

        portfolio[i] = portfolio[i-1] * (
            1 + daily_rets[i])

    final = CAPITAL * portfolio[-1]

    # Risk metrics
    cummax = np.maximum.accumulate(portfolio)
    dd = (portfolio - cummax) / cummax
    max_dd = dd.min() * 100
    sharpe = (np.mean(daily_rets[1:])
              / np.std(daily_rets[1:])
              * np.sqrt(252))

    print(f"${CAPITAL:,} -> ${final:,.0f} "
          f"(+{(final/CAPITAL-1)*100:.0f}%)")
    print(f"Sharpe:  {sharpe:.2f}")
    print(f"Max DD:  {max_dd:.1f}%")

    return common, portfolio, signals, aligned


if __name__ == "__main__":
    run_portfolio()
```

— *Robin*